

RoboRT: A Real-Time C++ Inference Engine for pi0.5 Vision-Language-Action Policies

liuwang
Baidu

Abstract

Vision-language-action (VLA) models such as pi0.5 have shown strong performance on robot manipulation, yet their inference latency remains the dominant bottleneck for real-time closed-loop control. We present RoboRT, a native C++ inference engine designed specifically for the flow-matching denoising schedule of pi0.5. Instead of adapting a generic large-language-model (LLM) runtime, RoboRT implements a hand-written GGUF reader that loads F32/BF16/quantized weights directly, a compute graph that is cached and rebuilt only when the input shape changes, and a member noise RNG that makes inference bit-exact reproducible. On an A10 GPU, RoboRT achieves **161 ms/step**, a **2.3×** speedup over the LeRobot PyTorch reference at 377 ms/step under the same observation and seed, while retaining 38/40 task success on the LIBERO benchmark. RoboRT is released under the Apache-2.0 license with a deterministic self-check tool that enables bit-exact comparison across implementations.

1 Introduction

Recent vision-language-action (VLA) policies, exemplified by the pi0 family [1], jointly model vision, language, and action in a single architecture and have become a de facto baseline for generalist robot manipulation. A VLA policy must run at the frequency of the control loop it drives: higher inference throughput directly translates to higher task success in dynamic manipulation. The practical target is on the order of 10 Hz per inference step; anything slower forces aggressive motion smoothing and degrades closed-loop behavior.

Despite the appeal of these models, deploying them at interactive rates is hard for three reasons. First, *reference implementations are slow*: the widely used LeRobot [3] stack runs pi0.5 at 377 ms/step on an A10, leaving little headroom for a reactive controller. Second, *generic LLM runtimes do not fit the workload*: mature inference engines are optimized for autoregressive token generation, whereas pi0.5 predicts action chunks through an iterative flow-matching denoising loop [4] with non-autoregressive, prefix-cached compute; grafting the model onto such a runtime carries a large amount of unused machinery and poorly matched scheduling. Third, *the deployment chain is fragmented*: vision encoding, prefix caching, noise scheduling, and action denormalization are usually bolted together from separate implementations, which makes results hard to reproduce and even harder to compare across codebases.

We address these limitations with **RoboRT**, a self-contained C++ inference engine for pi0.5. The engine is native rather than an adapter over an LLM framework: it reads weights with a hand-written GGUF reader, executes a compute graph that is cached across steps and rebuilt only when the input shapes change, and owns the noise RNG as engine state so that every prediction is bit-exact reproducible from a fixed seed. RoboRT builds only on generic compute primitives—the ggml graph runtime with its CPU/CUDA backends and the SigLIP vision tower—and carries no LLM-specific layers.

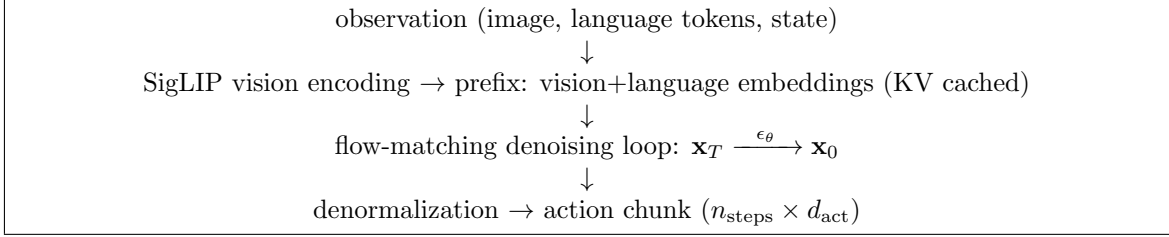


Figure 1: End-to-end data flow of RoboRT for one pi0.5 predict.

Our contributions are:

- **A native pi0.5 inference engine** (Sec. 3): hand-written GGUF reader, cached compute graph, and a flow-matching denoising loop, with no dependence on an LLM runtime.
- **A reproducibility mechanism** (Sec. 4): deterministic member RNG semantics and a self-check tool that dumps 1,600 floats bit-exact for cross-implementation regression.
- **Measured results** (Sec. 5): 161 ms/step on A10 (2.3× over LeRobot) and 38/40 task success on LIBERO under seed-42, consistent with the reference policy.

2 Related Work

pi0 and pi0.5. pi0 [1] is an open-source VLA built on the PaliGemma vision-language backbone with a flow-matching action expert; pi0.5 [2] extends it with a more capable backbone while keeping the same denoising interface. Both predict a chunk of actions (the suffix) by iteratively denoising Gaussian noise conditioned on vision, language, and proprioceptive state.

VLA inference stacks. LeRobot [3] provides the canonical PyTorch implementation of pi0 and pi0.5 and is the reference we compare against. Its latency is dominated by Python overhead and eager execution. Serving-oriented runtimes (e.g. SGLang [6]) bring batching and continuous KV cache but target autoregressive LLM serving rather than the non-autoregressive denoising loop of action experts.

Flow matching and diffusion inference. Flow matching [4] and diffusion models [5] share an iterative refinement schedule. Production inference for these models optimizes the number of steps, scheduling, and per-step compute—the same knobs RoboRT exposes—rather than token-level batching.

Graph runtimes. ggml [7] provides a dependency-free computation-graph runtime with CPU and CUDA backends. We reuse only its generic operators and implement all pi0.5-specific graph construction ourselves.

3 Method

Figure 1 gives the end-to-end data flow. The engine is organized as a single library (`vla-pi05`) with a public C++ API: a `Config` describing model dimensions, an `Inputs` struct carrying the observation, and a `predict` function returning a flat $n_{\text{steps}} \times d_{\text{act}}$ action chunk.

3.1 Hand-written GGUF reader

pi0.5 weights are distributed as HuggingFace safetensors. RoboRT converts them offline to GGUF with two Python scripts (`convert_pi05_to_gguf.py` and `convert_pi05_mmpoj_to_gguf.py`), then

loads them at startup with a hand-written reader over the ggml `gguf_*` API. The reader supports F32, BF16, and quantized dtypes and maps tensors directly into the compute graph without any model-level abstraction layer.

3.2 Cached compute graph

The inference graph has three shape-dependent inputs: the number of visual tokens, the number of language tokens, and the suffix length. RoboRT builds the graph once and caches it; the graph is rebuilt only when one of these shapes changes (e.g., a different language instruction). During the denoising loop the same graph is executed repeatedly, so the steady-state inference path has zero graph-construction overhead.

3.3 Flow-matching denoising loop

Given a noise tensor $\mathbf{x}_T \sim \mathcal{N}(0, I)$, the action expert is iterated as

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \Delta t \cdot \epsilon_\theta(\mathbf{x}_t, c), \quad (1)$$

where ϵ_θ is the denoising network conditioned on c (vision, language, and state) and $\Delta t = 1/n_{\text{steps}}$. The prefix (vision + language embeddings) is computed once and its key/value cache is reused across all denoising iterations. After the loop, the action chunk is denormalized inside the engine using the training statistics (quantile or mean/std per the GGUF metadata) and truncated to the real action dimension.

4 Reproducibility

Stochasticity in flow-matching inference comes from the noise sample, so reproducibility reduces to controlling the noise RNG. In RoboRT the RNG is engine state: it advances once per `predict`, and the environment variable `VLA_PI05_SEED` sets only the starting point of the sequence. The engine ships `vla-pi05-selfcheck`, a deterministic tool that runs a fixed seed and dumps the final prediction’s 1,600 floats. This enables bit-exact comparison across implementations: any regression in graph construction, operator selection, or normalization surfaces as a value diff. For faithful comparison, the server must be queried starting from a clean process—any extra request advances the noise sequence and shifts the marginal task outcomes.

The serving path additionally validates the input contract strictly: state length must equal the configured dimension, language token counts must lie in $[1, n_{\text{lang}}]$, the noise shape must match $\{0, n_{\text{suffix}} \times d_{\text{act}}\}$, and all float inputs must reject NaN/Inf.

5 Experiments

5.1 Setup

We evaluate on an A10 GPU (24 GB) with fp16 weights. The model is the pi0.5 checkpoint fine-tuned on LIBERO-90. Latency is measured as end-to-end `predict` time with a fixed seed (`VLA_PI05_SEED=42`). The LeRobot reference runs the same observation and seed in PyTorch.

Table 1: End-to-end latency per inference step on A10 (fp16).

Engine	Vision	Denoise	Other	Total
LeRobot (PyTorch)	—	—	—	377 ms
RoboRT (C++)	23.7 ms	126.9 ms	1.6 ms	161 ms

Table 2: LIBERO task success (seed-42, 1 episode per task).

Engine	Success	Failed tasks
LeRobot (reference)	38/40	{24, 29}
RoboRT (C++)	38/40	{24, 29}

5.2 Inference latency

Table 1 reports the end-to-end latency and its phase breakdown. RoboRT reaches 161 ms/step, a $2.3\times$ speedup over the LeRobot reference at 377 ms/step. The denoising phase (126.9 ms) dominates; vision encoding costs 23.7 ms.

5.3 LIBERO evaluation

We evaluate the policy with the LIBERO suite using one episode per task, a seed offset of 42, and no video recording. RoboRT achieves **38/40** task success; the two failures, tasks {24, 29}, match the LeRobot reference exactly, confirming the engine reproduces the reference policy’s behavior end to end.

6 Conclusion

We presented RoboRT, a native C++ inference engine for the pi0.5 vision-language-action policy. By matching the runtime to the flow-matching workload—hand-written GGUF loading, cached compute graphs, and deterministic noise—RoboRT delivers a $2.3\times$ latency reduction over the reference implementation while remaining bit-exact reproducible and behaviorally identical on LIBERO. The engine is released under Apache-2.0 with conversion scripts, a ZeroMQ/Protobuf serving layer, and a deterministic self-check tool.

References

- [1] M. Black et al. π_0 : A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.
- [2] S. Nasiriany et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv:2504.09754*, 2025.
- [3] R. Cadene et al. LeRobot: State-of-the-art machine learning for real-world robotics. <https://github.com/huggingface/lerobot>, 2024.
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *ICLR*, 2023.

- [5] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- [6] L. Zheng et al. SGLang: Efficient execution of structured language model programs. *arXiv:2312.07104*, 2023.
- [7] G. Gerganov et al. ggml: Tensor library for machine learning. <https://github.com/ggml-org/ggml>, 2023.